

Subject: Operating System

Unit : 5

Prof. Ashwani Mishra

Assistant Professor
AIML Department

CONTENTS

- Page Replacement Algorithms
- Thrashing & Demand Segmentation
- File System Interface
- File System Implementation
- Secondary Storage Structures
- Disk Management & Swap Space Management
- Disk Reliability & Stable Storage Implementation
- Protection
- Security

Page Replacement Algorithms

- User request for a page then the Operating System will check the page table for the required page, if it is present the page frame number is taken and it is added to the page offset to obtain the physical address of the frame.
- If the page is not in the page table, it is treated by the OS as a page fault. It is a trap to the OS, then the OS check for page on the Secondary memory, if it is available, then the page is brought into the main memory, and the page table is updated by setting the present or absent bit, and the instruction which requires the page is restarted. The process is called page fault handling.
- If the page is present on the disk and there is no free memory frame available in the main memory. Then one of the pages has to be removed from the memory and the required page has to be brought into the main memory, its a problem for the OS, then a number of policies are required the OS needs the page replacement algorithms.
- The following are to be considered like FIFO, Optimal, LRU, Second chance, Clock, Most Frequently used, Least Frequently used, etc.

FIFO Algorithm

- The simplest page-replacement algorithm is a FIFO algorithm. A FIFO replacement algorithm associates with each page the time when that page is chosen.
- It is not necessary to record the time when a page is brought in.
- We can create a FIFO queue to hold all pages in memory. We replace the page at the head of the queue.
- When a page is brought into memory, we insert it at the tail of the queue.

Continued ...

- The first three references (7, 0, 1) cause page faults, and are brought into these empty frames.
- The Next reference (2) replaces page 7, because page 7 was brought in first.
- Next reference (0) and 0 is already in memory; we have no fault for this reference.
- The first reference to 3 results in page 0 being replaced, since it was the first of the 3 pages in memory to be brought in.
- This replacement means that the Next reference, to 0, will fault.
- Page 1 is replaced by page 0.
- This process continues as shown in the below figure.

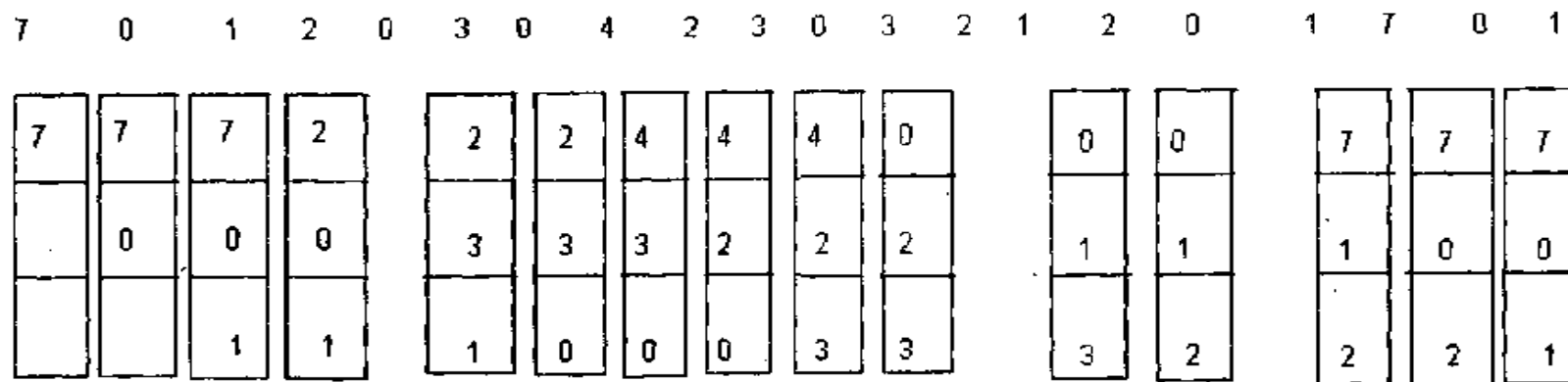


Fig: FIFO Page-replacement algorithm.

Continued...

Optimal Algorithm

- Belady's anomaly shows the search for an optimal page replacement algorithm. An optimal page replacement algorithm has the lowest page fault rate of all algorithms.
- An optimal algorithm will never suffer from Belady's anomaly. An optimal page replacement algorithm exists, and has been called OPT or MIN. It simply replaces the page that will not be used for the longest period of time.

LRU Algorithm

- Replace the page that has not been used for longest period of time. This approach is the least recently used(LRU) algorithm.

Continued...

Counting Algorithms

- There are many other algorithms that can be used for page replacement. For example we can keep the counter of the number of references that have been made to each page, and develop the following two schemes:
- **LFU Algorithm:** The least frequently used (LFU) page-replacement algorithm requires that the page with the smallest count be replaced. The reason for this selection is that an actively used page should have a large reference count. This algorithm suffers from the situation in which a page is used heavily during the initial phase of a process, but then is never used again.
- **MFU Algorithm:** The most frequently used (MFU) page-replacement algorithm is based on the argument that the page with the smallest count was probably just brought in and has yet to be used.

Continued...

Page Buffering Algorithm

- Systems commonly keep a pool of free frames. When a page fault occurs, a victim frame is chosen as before. However, that desired page is read into a free frame from the pool before the victim is written out. This procedure allows the process to restart as soon as possible, without waiting for the victim page to be written out.

Allocation of Frames

- The issue here is how to allocate frames amongst processes. Under pure demand paging, all page faults would get free frames if available. If there are no free frames, a page replacement algorithm would be used to swap out a process to get free frame.

Allocation Algorithm

- **Equal Allocation:** Every process is given an equal number of frames.
Example: If there are m frames, to be divided amongst n processes then an equal share of m/n frames are allocated. If there are 70 frames and 10 processes, then each process gets 7 frames each and the leftover 5 frames could be set to free frame pool.
- **Proportional Allocation:** In equal allocation of frames, many frames may not be used by the process and hence wasted. In proportional allocation, a process is allocated frames according to its requirement.

Thrashing and Demand Segmentation

- The number of frames allocated to a low-priority process falls below the minimum number required by the computer architecture; we must suspend that process's execution.
- We should then page out its remaining pages, freeing all its allocated frames. This provision introduces a swap-in, swap-out level of intermediate CPU scheduling.
- In fact, look at any process that does not have "enough" frames. If the process does not have the number of frames it needs to support pages in active use, it will quickly page-fault. At this point, it must replace some page. However, since all its pages are in active use, it must replace a page that will be needed again right away. Consequently, it quickly faults again, and again, and again, replacing pages that it must bring back in immediately. This high-paging activity is called thrashing. A process is thrashing if it is spending more time paging than executing.

Cause of Thrashing:

- Thrashing results in severe performance problems. Consider the following scenario, which is based on the actual behaviour of early paging systems.
- The operating system monitors CPU utilisation. If CPU utilisation is too low, we increase the degree of multiprogramming by introducing a new process to the system.
- A global page-replacement algorithm is used; it replaces pages without regard to the process to which they belong. Now, suppose that a process enters a new phase in its execution and needs more frames. It starts faulting and taking frames away from other processes.

Continued...

Working-Set Model

- The working-set model is based on the assumption of locality. This model uses a parameter A to define the working-set window. The idea is to examine the most recent A page references.
- A page references. The set of pages in the most recent A page references is the working set. If a page is in, active use, it will be in the working set. If it is no longer being used, it will drop from the working set A time units after its last reference. Thus, the working set is an approximation of the program's locality.

Page-Fault Frequency

- The working-set model is successful, and knowledge of the working set can be useful for pre-paging, but it seems a clumsy way to control thrashing. A strategy that uses the page-fault frequency (PFF) takes a more direct approach.
- The specific problem is how to prevent thrashing. Thrashing has a high page-fault rate. Thus, we want to control the page-fault rate when it is too high; we know that the process needs more frames.

Demand Segmentation

- Although demand paging is generally considered the most efficient virtual memory system, a significant amount of hardware is required to implement it.
- When this hardware is lacking, less efficient means are sometimes devised to provide virtual memory. A case in ease demand segmentation.

File System Interface

The file system consists of two distinct parts:

- A collection of files, each storing related data, and
- A directory structure, which organises and provides information about all the files in the system.'

File Concept

- A file is a named collection of related information that is recorded on secondary storage. A file is a sequence of bits, bytes, lines, or records whose meaning is defined by the file's creator and user.

File Attributes

- A file has certain other attributes, which vary from one operating system to another, but typically consists of these:
- **Name:** The symbolic file name is the only information kept in human-readable form.
- **Type:** This information is needed for those systems that support different types.
- **Location:** This information is a pointer to a device and to the location of the file on that device.
- **Size:** The current size of the file (in bytes, words, or blocks), and possibly the maximum allowed size are included in this attribute.
- **Protection:** Access control information controls who can do reading, writing, executing, and so on.
- Time, Date and user identification: This information may be kept from creation, last modification, and last use.

Continued...

- **Writing a File:** A system call is made specifying both the name of the file and the information to be written to the file. A write pointer is maintained which points to the location from where the next write is to take place. The write pointer must be updated whenever write occurs.
- **Reading a File:** A system call is issued that specifies the name of the file and where in the memory the next block of the file should be put for reading. A reader pointer is maintained that points to the next block to be read.
- **Repository with in a File:** The directory is searched for the appropriate entry, and the current file position is set to a given value. Repositioning with in a file does not need to involve any actual I/O. This file operation is also known as a file seek.
- **Deleting a File:** To delete a file, we search the directory for the named file. Having found the associated directory entry, we release all file space (so that it can be reused by other files) and erase the directory entry.
- **Truncating a File:** To erase the contents of the file but keep the attributes same, we can truncate the file, i.e., the attributes of the file remain same but the length of the file is reset to zero.
- **File Pointer:** On systems that do not include a file offset as part of the read and write system calls, the system must track the last read/write location as a current file position pointer. This pointer is unique to each process operating on the file, and therefore must be kept separate from the on disk file attributes.

Continued...

File Types

- Types is to include the type as part of The name and extension, usually a common technique for implementing file the file name.
- The name is split into two parts, separated by a period character.

FILE TYPE	USUALEXTENSION	FUNCTION
Executable	exe, com, bin, or none	Ready-to-run machine language program
Object	obj, o	Compiled, machine language, not linked
Source code	c, p, pas, f77, asm, a	Source code in various languages
Batch	bat, sh	Commands to the command interpreter.
Text	txt, doc	Textual data, documents
Word processor	wp, tex, rrf, etc	Various word processor formats
Library	lib, a	Libraries of routines for programmers
Print or view	ps, dvi, gif	ASCII or binary file into format for printing or viewing
Achive	arc, zip, tar	Related files grouped into one file sometimes compressed, for archiving or storage

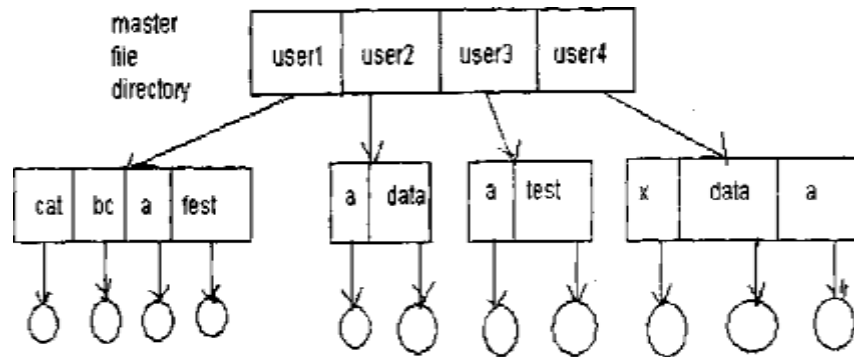
Directory Structures

When considering a particular directory structure, the operation that are to be performed on a directory are kept in mind.

- **Search for a file:** We need to be able to search a directory structure to find the entry or a particular file. Since files have symbolic names, and similar names may indicate a relationship between files, we may want to be able to find all files whose names match a particular pattern.
- **Create a File:** New files need to be created and added to the directory.
- **Delete a File:** When a file is no longer needed, we want to remove it from the directory.
- **List a Directory:** To be able to list the files in a directory, and the contents of the directory entry for each file in the list.
- **Rename a File:** Because the name of a file represents its contents to its users, the name must be changeable when the contents or use of a file changes. Renaming a file may also allow its position within the directory structure to be changed.
- **Traverse the file system.** It is useful to be able to access every directory, and every file within a directory structure.

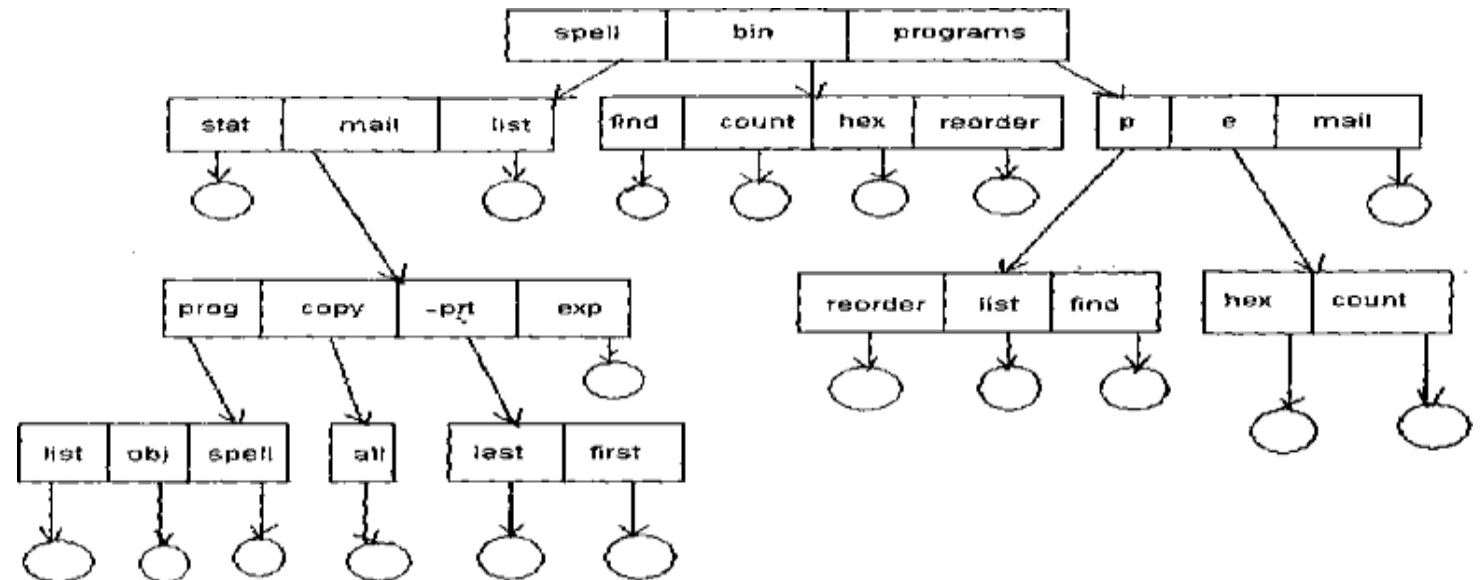
Continued...

Two - Level Directory



Tree-structure Directories

- The natural generalization from a two-level directory to a tree level directory is to extend the directory structure to a tree of arbitrary height.



- The major disadvantage to a single level directory is the confusion of file names between different users. The standard solution is to create a separate directory for each user.

Continue...

Types of Access

Protection mechanism provide controlled access by limiting the types of file access that can be made, Access is permitted or denied depending on several factors, one of which is the type of access requested. Several different types of operations may be controlled:

- **Read:** Read from the file.
- **Write:** Write or rewrite the file
- **Execute:** Load the file into memory and execute it.
- **Append:** Write new information at the end of the file.
- **Delete:** Delete the file and free its space for possible reuse.
- **List:** List the name and attributes of the file.

Other operations, such as renaming, copying, or editing the file, may also be controlled.

File System Implementation

File system Organization

- To provide an efficient and convenient access to the disk, the operating system imposes a file system to allow the data to be stored, located and retrieved easily.
- A file system poses two quite different design problems. The first problem is defining how the file system should look to the user. This task involves the definition of a file and its attributes, operations allowed on a file, and the directory structure for organising the files next, algorithms and data structures must be created to map the logical file system onto the physical secondary storage devices.

File allocation methods

The major methods of allocating disk space in wide use are:

- Contiguous, linked, and indexed.
- Contiguous Allocation
- The contiguous allocation method requires each file to occupy a set of contiguous blocks on the disk. Disk addresses define a linear ordering on the disk.
- Contiguous allocation of files is defined by the disk address and length (in block units) of the first block. If the file is n blocks long, and starts at location b , then it occupies blocks $b, b+1, b+2, \dots, b+n-1$. First fit and best fit are the most common strategies used to select a free hole from a set of available holes.

File System Implementation

Linked Allocation

- Linked allocation solves all the problems of contiguous allocation. With linked allocation, each file is a linked list of disk blocks; the disk blocks may be scattered anywhere on the disk. To create a new file, we simply create a new entry in the directory. With linked allocation, each directory entry has a pointer to the first disk block of the file. The pointer is initialised to nil (the end-of-list) pointer value) to signify an empty file.

Directory Implementation

- **Linear List**
- The simplest method of implementing a directory is to use a linear list of file names with pointers to the data blocks. This method is simple to program but is time-consuming to execute. To create a new file, we must first search the directory to be sure that no existing file has the same name. Then, we add a new entry at the end of the directory.
- **Hash Table**
- Another data structure that has been used for a file directory is a hash table. In this method, a linear list stores the directory entries, but a hash data structure is also used. The hash table takes a value computed from the file name and returns a pointer to the file name in the linear list. Therefore, it can greatly decrease the directory search time.

Secondary Storage Structures

- Disk drives are the major secondary storage I/O devices on most computer systems, along with tapes. The operating system maps the logical address generated by the CPU onto the physical memory, in turn, they are program addresses.
- The programs are already stored on secondary storage devices like disks and tapes. So, there is a movement of data from the disk onto the main memory, and from the main memory to the disk.

Shortest Seek Time-First: (SSTF)

- It seems reasonable to service all the requests close to the current head position before moving the head far away to service other requests. This assumption is the basis for the shortest-seek-time-first (SSTF) algorithm. The SSTF algorithm selects the request with the minimum seek time from the current head position. Since seek time increases with the number of cylinders traversed by the head, SSTF chooses the pending request closest to the current head position.

SSTF Disk Scheduling

- Suppose that we have two requests in the queue, for cylinders 14 and 186, and while servicing the request from 14, a new request near 14 arrives. This new request will be serviced next, making the request at 186 wait. While this request is being serviced, another request close to 14 could arrive. In theory, a continual stream of requests near one another could arrive, causing the request for cylinder 186 to wait indefinitely.

Disk Management and Swap Space Management

The operating system is responsible for several other aspects of disk management, too.

Disk Formatting

- Before using any disk, it must be formatted. It must be divided into sectors that the disk controller can read and write. This process is called low-level formatting or physical formatting.

Boot Block

- The small program, which initialises all the above aspects of the system, is called 'Bootstrap loader', and the process is called 'system booting'.

Bad Blocks

- Disks have moving parts and small tolerances they are prone to errors. The disk controller handles the problem regarding the bad blocks and sectors.

Swap Space Management

- Swap-space management is another low-level task of the operating system. Virtual memory uses disk space, as an extension of main memory, i.e. the logical address space imagined by the user is generally nothing but the disk.
- The programs are stored on the disk, and later, they were brought into the main memory as piecewise manner.

Disk Reliability And Stable Storage Implementation

- Disk is the least reliable components of a system. Recovering from a disk crash may take hours, and backup data to be reloaded for restoring the data.
- Keeping a large number of disks in parallel will improve the storage capacity and improve the retrieval speeds. This organisation is called a “redundant array of independent disks” (RAID). This improves the reliability of data by redundantly storing the data.

Stable-Storage Implementation

- To obtain stable storage, we need to replicate the needed information on multiple storage devices with independent failure modes.
- We require that, whenever a failure occurs during the writing of a block, the system detects it and invokes a recovery procedure to restore the block to a consistent state. To do this, the system must maintain two physical blocks for each logical block.

An output operation is executed as follows:

- Write the information onto the first physical block.
- When the first write completes successfully, write the same information onto the second physical block.
- Declare, the operation complete only after the second write completes successfully.

Protection

Goals of Protection

- Protection was originally conceived as an adjunct to multiprogramming operating systems, so that users might safely share a common logical name space, such as a directory- of files, or share a common physical name space, such as memory.
- Modern protection concepts have evolved to increase the integrity and reliability of any complex system that makes use of shared resources.
- A computing system needs to be protected against mischievous, intentional violation of an access restriction by untrustworthy users to maintain reliability of the system. Protection can improve reliability by detecting latent errors at the interfaces between component subsystems.
- Early detection of interface errors can often prevent contamination of a healthy subsystem by a malfunctioning subsystem.
- A protection-oriented system provides means to distinguish between authorised and unauthorised usage.

Security

- Security violations (or misuse) of the system can be categorised as intentional (malicious) or accidental. It is easier to protect against accidental misuse than against malicious misuse. For the most part, protection mechanisms are the core of protection from accidents. The following list includes forms of accidental and malicious security violations. To protect a system, we must take security measures at four levels:
 - **Physical:** The site or sites containing the computer systems must be physically secured against armed or surreptitious entry by intruders. Both the machine rooms and the terminals or workstations that have access to the machines must be secured.
 - **Human:** Authorizing users must be done carefully to assure that only appropriate users have access to the system. Even authorised users, however, may be "encouraged" to let others use their access (in exchange for a bribe, for example).
 - **Operating System:** The system must protect itself from accidental or purposeful security breaches. A runaway process could constitute an accidental denial-of-service attack. A query to a service could reveal passwords. A stack overflow could allow the launching of an unauthorised process. The list of possible breaches is almost endless.

Security

Types of Viruses

Program Threats

- Writing a program that creates a breach of security, or causing a normal process to change its behavior and create a breach, is a common goal of crackers.

Trojan Horse

- A variation of the Trojan horse is a program that emulates a login program. An unsuspecting user starts to log in at a terminal and notices that he has apparently mistyped his password. He tries again and is successful. What has happened is that his authentication key and password have been stolen by the login emulator, which was left running on the terminal by the thief.

Trap Door

- The designer of a program or system might leave a hole in the software that only she is capable of using. For instance, the code might check for a specific user ID or password, and it might circumvent normal security procedures. Programmers have been arrested for embezzling from banks by including rounding errors in their code and having the occasional half-cent credited to their accounts. This account crediting can add up to a large amount of money, considering the number of transactions that a large bank executes.

THANK YOU